

CS-3510-B Problem Set 3

Elijah Martin McCauley

TOTAL POINTS

59 / 60

QUESTION 1

Problem 1 20 pts

1.1 Part A 8 / 8

✓ - 0 pts a) Correct

![Screen_Shot_2023-02-

11_at_4.20.34_PM.png](/files/2c22a621-c531-4f8c-ab06-e465adc41c79)

![Screen_Shot_2023-02-

11_at_5.17.23_PM.png](/files/293146c5-9b91-416e-88c4-2e27621def63)

- 1 pts a) One or two labels incorrect
- 2 pts a) Three to six labels incorrect
- 4 pts a) More than six labels incorrect
- 5 pts a) Missing answer
- 0 pts b) Correct
- 2 pts b) Incorrect/ Missing Explanation
- 3 pts b) Incorrect/ Missing Answer

1.2 Part B 12 / 12

✓ - 0 pts a) Correct

![Screen_Shot_2023-02-

18_at_7.01.51_PM.png](/files/561148dc-cb3b-4b08-a55f-3117844e3020)

- 1 pts a) One to three orderings incorrect
- 3 pts a) More than three orderings wrong

- 4 pts a) Incorrect/ Missing Answer

✓ - 0 pts b) Correct

![Screen_Shot_2023-02-

18_at_7.01.59_PM.png](/files/6ee1ac38-9ad2-4d14-bcab-b6ae5b346f30)

- 1 pts b) One to two incorrect
- 3 pts b) More than two incorrect
- 4 pts b) Missing answer
- 0 pts c) Correct (3)
- 4 pts c) Incorrect/ Missing answer

QUESTION 2

Problem 2 20 pts

2.1 Part A 10 / 10

✓ - 0 pts Correct

- 2 pts No graph definition provided/ Minor Error in Algorithm

- 5 pts Major Error/ Mostly Incorrect/

Suboptimal Algorithm

- 5 pts Pseudo-code provided, but algorithm is missing.

- 2 pts Incorrect/ Missing Runtime Analysis

- 3 pts Incorrect/ Missing Proof of Correctness

- 10 pts Incorrect/ Missing answer

2.2 Part B 10 / 10

✓ - 0 pts Correct

- **2 pts** Minor Error in Algorithm
- **5 pts** Major Error/ Mostly Incorrect/

Suboptimal Algorithm

- **5 pts** Pseudo-code provided, but algorithm is missing.

- **2 pts** Incorrect/ Missing Runtime Analysis
- **3 pts** Incorrect/ Missing Proof of Correctness
- **10 pts** Incorrect/ Missing answer

QUESTION 3

3 Problem 3 19 / 20

- **0 pts** a) Correct
- **3 pts** a) No graph definition provided/ Minor

Error in the Algorithm

- **7 pts** a) Major Error/ Mostly Incorrect/

Suboptimal Algorithm

- **7 pts** a) Pseudo-code provided, but algorithm is missing.

- **5 pts** (a) Partial Correct
- **3 pts** a) Incorrect/ Missing Runtime Analysis
- **5 pts** a) Incorrect/ Missing Proof of

Correctness

- **15 pts** a) Incorrect/ Missing answer
- **0 pts** b) Correct

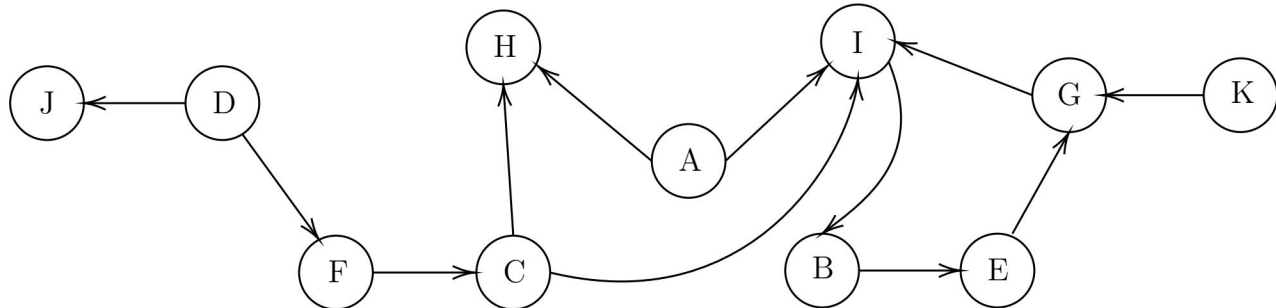
✓ - **1 pts** b) *Inefficient Approach*

- **2 pts** b) Partially correct
- **5 pts** b) Incorrect/ Missing answer

Problem 1

(20 points)

Consider the graph below:



Part A

For this part, in the event of a tie, choose the node that is alphabetically first.

- Run DFS on this graph and list the pre and post labels of each vertex.
- Use the labels from (a.) to topologically sort this graph. List the vertices in topological order.

Solution: a.)

a 1, 12
b 5, 10
c 13, 14
d 15, 20
e 6, 9
f 16, 17
g 7, 8
h 2, 3
i 4, 11
j 18, 19
k, 21, 22
b.)

It is not possible to topologically sort this graph because you can only run a topological sort on directed acyclic graphs and this graph has a cycle (i, b, e, g). This cycle means that you cannot put the vertices in a linearized order which is the goal of toposort.

Part B

For this part, we run the SCC algorithm on the above graph. When running DFS on the reverse graph, use the alphabetical order of vertices.

- Write the strongly connected components in the order you found them.

1.1 Part A 8 / 8

✓ - 0 pts a) Correct

![Screen_Shot_2023-02-11_at_4.20.34_PM.png](/files/2c22a621-c531-4f8c-ab06-e465adc41c79)

![Screen_Shot_2023-02-11_at_5.17.23_PM.png](/files/293146c5-9b91-416e-88c4-2e27621def63)

- 1 pts a) One or two labels incorrect
- 2 pts a) Three to six labels incorrect
- 4 pts a) More than six labels incorrect
- 5 pts a) Missing answer
- 0 pts b) Correct
- 2 pts b) Incorrect/ Missing Explanation
- 3 pts b) Incorrect/ Missing Answer

- b.) Label all the sink components and all the source components.
- c.) Give the minimum number of edges that must be added to the graph above to make it strongly connected.

Solution: a.)

J, H, BEGI, K, C, F, D, A

b.)

Sinks: H, J, IBEG

Sources: A, D, K c.)

The minimum number of edges that must be added is 3. An example of this is adding H $\rightarrow$ A, J $\rightarrow$ K, and I $\rightarrow$ D because this eliminates all the sinks and sources by adding an outgoing edge from the sinks and an incoming edge to the sources.

1.2 Part B 12 / 12

✓ - 0 pts a) Correct

!/[Screen_Shot_2023-02-18_at_7.01.51_PM.png](/files/561148dc-cb3b-4b08-a55f-3117844e3020)

- 1 pts a) One to three orderings incorrect

- 3 pts a) More than three orderings wrong

- 4 pts a) Incorrect/ Missing Answer

✓ - 0 pts b) Correct

!/[Screen_Shot_2023-02-18_at_7.01.59_PM.png](/files/6ee1ac38-9ad2-4d14-bcab-b6ae5b346f30)

- 1 pts b) One to two incorrect

- 3 pts b) More than two incorrect

- 4 pts b) Missing answer

- 0 pts c) Correct (3)

- 4 pts c) Incorrect/ Missing answer

Problem 2

(20 points)

Part A

(10 points) Let $G = (V, E)$ be a weighted Directed Acyclic Graph (DAG). Design an efficient algorithm to find the shortest path from a source vertex s to all other vertices. Make sure to provide proof of the correctness of your algorithm and the time complexity.

Note: You cannot use Dijkstra's Algorithm as a black-box for this problem.

Solution: Algorithm: First we run a topological sort on the graph G . This will give us the nodes in linearized order where each node is increasingly far away from s (in terms of number of nodes not necessarily distance away). For all nodes n in the graph, we instantiate the distance to them as infinity. We set the distance to s as 0. We can then loop through the nodes in the graph in linearized order starting at s . For each node, get its edges. For each of its edges, get the distance from s to n , and add it to the length of the edge from n to its child m . If s to n plus n to m is less than the current distance recorded from s to m (starting at infinity), update that to the new distance. Once we have gone through each node in linearized order, we will have the shortest weighted distance to each one.

Proof of Correctness: This algorithm works because after running toposort, you have the nodes in linearized order meaning you reach each node after you reach the node before it in the list. So, if you start from the only source, you first get the distances to the nodes one node away from the source, then you will go to the nodes two away and add the edge length to the distance from the start to the parent node. If there is multiple ways to reach a node, the algorithm checks if the distance of the current path is less than the best distance and if it is then it updates to the new distance.

Time Complexity: The runtime for this algorithm would be $\mathcal{O}(V + E)$ because Toposort is $\mathcal{O}(V + E)$ and the work of the loop afterward is also $\mathcal{O}(V + E)$ because you check every vertex once and every edge once.

Part B

(10 points) Suppose a wedding is hosted at your house and you have invited n guests. However, each of the guests has a list of close friends and relatives with that they must sit at the same table. Since some of these tables might have a lot of people sitting at them, we want to make sure the tables we buy are large enough. Design an algorithm to determine the maximum number of people sitting at any one table. Make sure to provide proof of the correctness of your algorithm and the time complexity.

Solution:

Algorithm: We first need to create a graph of all the guests where each person points to

2.1 Part A 10 / 10

✓ - 0 pts *Correct*

- 2 pts No graph definition provided/ Minor Error in Algorithm
- 5 pts Major Error/ Mostly Incorrect/ Suboptimal Algorithm
- 5 pts Pseudo-code provided, but algorithm is missing.
- 2 pts Incorrect/ Missing Runtime Analysis
- 3 pts Incorrect/ Missing Proof of Correctness
- 10 pts Incorrect/ Missing answer

Problem 2

(20 points)

Part A

(10 points) Let $G = (V, E)$ be a weighted Directed Acyclic Graph (DAG). Design an efficient algorithm to find the shortest path from a source vertex s to all other vertices. Make sure to provide proof of the correctness of your algorithm and the time complexity.

Note: You cannot use Dijkstra's Algorithm as a black-box for this problem.

Solution: Algorithm: First we run a topological sort on the graph G . This will give us the nodes in linearized order where each node is increasingly far away from s (in terms of number of nodes not necessarily distance away). For all nodes n in the graph, we instantiate the distance to them as infinity. We set the distance to s as 0. We can then loop through the nodes in the graph in linearized order starting at s . For each node, get its edges. For each of its edges, get the distance from s to n , and add it to the length of the edge from n to its child m . If s to n plus n to m is less than the current distance recorded from s to m (starting at infinity), update that to the new distance. Once we have gone through each node in linearized order, we will have the shortest weighted distance to each one.

Proof of Correctness: This algorithm works because after running toposort, you have the nodes in linearized order meaning you reach each node after you reach the node before it in the list. So, if you start from the only source, you first get the distances to the nodes one node away from the source, then you will go to the nodes two away and add the edge length to the distance from the start to the parent node. If there is multiple ways to reach a node, the algorithm checks if the distance of the current path is less than the best distance and if it is then it updates to the new distance.

Time Complexity: The runtime for this algorithm would be $\mathcal{O}(V + E)$ because Toposort is $\mathcal{O}(V + E)$ and the work of the loop afterward is also $\mathcal{O}(V + E)$ because you check every vertex once and every edge once.

Part B

(10 points) Suppose a wedding is hosted at your house and you have invited n guests. However, each of the guests has a list of close friends and relatives with that they must sit at the same table. Since some of these tables might have a lot of people sitting at them, we want to make sure the tables we buy are large enough. Design an algorithm to determine the maximum number of people sitting at any one table. Make sure to provide proof of the correctness of your algorithm and the time complexity.

Solution:

Algorithm: We first need to create a graph of all the guests where each person points to

their close friends and family and their friends and family point back to them. We then run a version of the SCC algorithm which returns a list containing the strongly connected components. Each strongly connected component is equivalent to a table at the wedding because if anyone is close friends with another person they must sit at the same table and you can therefore reach anyone at the table through friends of friends. Once you have the list of strongly connected components, you can simply loop through the list and count the number of nodes in each. Whatever is the maximum nodes in a scc is the maximum number of people at a single table.

Proof of Correctness: Every guest and their list of close friends and relatives are already strongly connected because their friends and family point to them meaning anyone on their list can reach anyone else on the list. If someone from that first list is on a different list as well, that whole other list can be reached by everyone on the first list, so that table must also sit with the first table and is part of the same strongly connected component. Therefore the strongly connected components we find are the tables because everyone is a mutual friend of everyone else even if it is multiple degrees away. The different SCCs are when no one in a given group/set of groups is friends with anyone outside their current table. This means the greatest number of people at a table is the largest SCC.

Time Complexity: The run time for this algorithm would be $\mathcal{O}(V + E)$ because you run the SCC algorithm which is $\mathcal{O}(V + E)$ and looping through the list of SCCs is $\mathcal{O}(V)$ which gets absorbed into SCC algorithm's $\mathcal{O}(V + E)$.

2.2 Part B 10 / 10

✓ - 0 pts *Correct*

- 2 pts Minor Error in Algorithm
- 5 pts Major Error/ Mostly Incorrect/ Suboptimal Algorithm
- 5 pts Pseudo-code provided, but algorithm is missing.
- 2 pts Incorrect/ Missing Runtime Analysis
- 3 pts Incorrect/ Missing Proof of Correctness
- 10 pts Incorrect/ Missing answer

Problem 3

(20 points)

Suppose Mr. Kartik is hosting a party with international delegates, where all the guests speak unique languages. However, each guest can understand (but cannot speak) some of the other languages spoken at the party. In this way, two guests that don't understand each other can have a conversation by using another guest (or more) as a "translator". You're given a list of guests and which languages they understand.

As an example, if A is trying to communicate with C , and B understands A , and C understands B , then C can effectively understand A . However, A can't necessarily understand C .

- a.) (15 points) You want to form the minimum number of tables so that everyone at a table is able to converse with everyone else at the table (that is, for any two people at the table, they can both understand each other). Design an efficient algorithm for this. Make sure to provide proof of the correctness of your algorithm and the time complexity.
- b.) (5 points) Now you want to pass a message around to as many tables as possible, by starting it at some table (remember that some guests can understand guests at other tables). Explain your approach to get the maximum number of tables your message can reach.
- Note:* No need to provide a formal algorithm, but a small paragraph explaining your approach is sufficient.

Solution:

a.)

Algorithm: We first need to create a directed graph where each node is a person which points to the people who can understand them. From this, we can run a modified SCC algorithm which outputs the number of strongly connected components. This number will be the minimum number of tables needed so that everyone at the table will understand each other.

Proof of correctness: This will always return the minimum number of tables because of the nature of strongly connected components. To be a strongly connected component, you can get to every node from every other node. In this context, where directed edges are equivalent to someone understanding another person's language, it means that everyone in that group can communicate with everyone else by going around in a cycle of translation. If B understands A , A understands C , and C understands B then they can all communicate and be a SCC, if C cannot understand B then they would not be able to all reach each other and therefore not be an SCC.

Time Complexity: The time complexity would be $\mathcal{O}(V + E)$ because the SCC algorithm runs in $\mathcal{O}(V + E)$.

b.)

In order to pass a message around to as many tables as possible, you would want to construct a metagraph of the tables where the directed connections are when a person at one table can understand another table but not communicate back. Once we have this metagraph, we find the sources and run a modified dfs on each source which counts the nodes reachable by

them. The one that reaches the most nodes is the table we start with.

3 Problem 3 19 / 20

- 0 pts a) Correct
- 3 pts a) No graph definition provided/ Minor Error in the Algorithm
- 7 pts a) Major Error/ Mostly Incorrect/ Suboptimal Algorithm
- 7 pts a) Pseudo-code provided, but algorithm is missing.
- 5 pts (a) Partial Correct
- 3 pts a) Incorrect/ Missing Runtime Analysis
- 5 pts a) Incorrect/ Missing Proof of Correctness
- 15 pts a) Incorrect/ Missing answer
- 0 pts b) Correct
- ✓ - 1 pts b) *Inefficient Approach*
- 2 pts b) Partially correct
- 5 pts b) Incorrect/ Missing answer

Name: Elijah McCauley

CS 3510: Design & Analysis of Algorithms

01/26/2023

Problem Set 3: BFS, DFS, Topological Sort & SCC

Abraham Ladha

Due: 02/09/2023 11:59pm

- Please TYPE your solutions using Latex or any other software. Handwritten solutions *won't* be accepted.
- Your solutions must be in plain English and mathematical expressions.
- Unless otherwise stated, the fastest (and correct) algorithms will receive more credit. If we ask for a specific running time, a correct solution achieving it will receive full credit even if a faster solution exists.
- Do not use pseudocode. Your answer will receive zero credit even if the pseudocode is correct.
- You can use the following algorithms as black-box for this problem set i.e. you don't have to explain the algorithm while using it. For example, you can write "Using DFS, compute the post labels":
 - BFS
 - DFS
 - Explore(v)
 - Toposort
 - SCC
 - Construct a reverse graph
 - Construct the metagraph
- When we say "design an efficient algorithm", we are asking for a detailed description of the algorithm in English, i.e., without any code or pseudocode. Fastest (and correct) solutions are worth more credit.